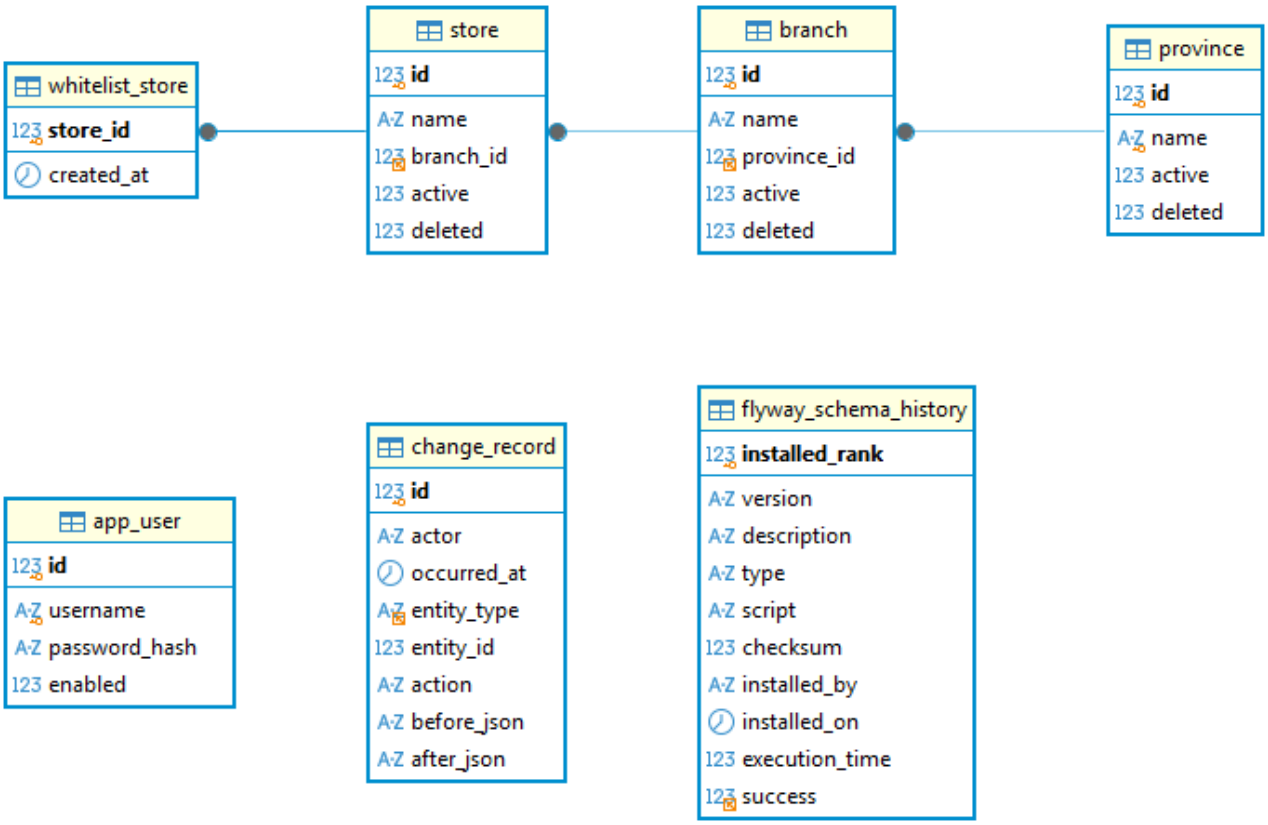
Database Design

MySQL schema, relationships and query behavior for the fictional master-data service.

Entity relationship diagram

This diagram is exported from DBeaver's Diagram view of the local **hlu_assignment** schema. Connector lines show actual database foreign keys.



DBeaver diagram of the application tables and Flyway migration history.

Reading the relationships

One province can have many branches. Each branch belongs to one province. One branch can have many stores. Each store belongs to one branch. A store can have at most one whitelist entry.

change_record.actor stores the authenticated username as a historical text snapshot. It is not a foreign key to **app_user**, so DBeaver correctly shows no connector. **flyway_schema_history** is managed separately by Flyway.

Tables and key fields

The migration is the authoritative schema definition. Primary keys are abbreviated PK and foreign keys FK below. All sample records are fictional.

Table	Columns and constraints
province	id BIGINT PK; name VARCHAR(120) unique; active and deleted BOOLEAN.
branch	id BIGINT PK; name VARCHAR(120); province_id BIGINT FK; active and deleted BOOLEAN.
store	id BIGINT PK; name VARCHAR(120); branch_id BIGINT FK; active and deleted BOOLEAN.
whitelist_store	store_id BIGINT PK and FK; created_at DATETIME(6). Membership is unique per store.
app_user	id BIGINT PK; username VARCHAR(100) unique; password_hash VARCHAR(100); enabled BOOLEAN.
change_record	id BIGINT PK; actor VARCHAR(100); occurred_at DATETIME(6); entity_type and action VARCHAR(30); entity_id BIGINT; before_json and after_json nullable TEXT.
flyway_schema_history	Flyway migration metadata. This table is maintained by Flyway.

Example: whitelist membership

```
CREATE TABLE whitelist_store (
    store_id BIGINT NOT NULL PRIMARY KEY,
    created_at DATETIME(6) NOT NULL,
    CONSTRAINT fk_whitelist_store FOREIGN KEY (store_id)
        REFERENCES store(id)
) ENGINE=InnoDB;
```

The primary key prevents duplicate entries. The foreign key prevents a membership from referring to a missing store. Removing a membership does not remove the store.

IDs are generated by MySQL except for the whitelist key, which reuses the store ID. Visibility flags distinguish inactive records from soft-deleted records. Timestamps are stored as UTC instants.

Search and visibility

The service first finds active, non-deleted provinces whose names contain the supplied text. The following is a readable SQL equivalent of the store page query, with two example province placeholders. Actual SQL is generated from the repository query.

```
SELECT s.id, s.name, b.id, b.name, p.id, p.name
FROM store s
JOIN branch b ON b.id = s.branch_id
JOIN province p ON p.id = b.province_id
WHERE s.active = TRUE AND s.deleted = FALSE
  AND b.active = TRUE AND b.deleted = FALSE
  AND p.active = TRUE AND p.deleted = FALSE
  AND (p.id IN (?, ?) OR EXISTS (
      SELECT 1 FROM whitelist_store w
      WHERE w.store_id = s.id
  ))
ORDER BY s.id ASC
LIMIT ? OFFSET ?;
```

The database applies visibility, combines matching provinces with whitelist membership, orders the result and returns the requested page. EXISTS avoids duplicating stores that satisfy both conditions. A separate count query supplies totalElements and totalPages.

Indexes and performance

The unique province-name index enforces uniqueness. Branch and store indexes include their parent ID and visibility flags. The whitelist primary key supports membership checks; idx_change_entity supports entity history lookup.

Substring matching has a leading wildcard and can scan province names. Deep offsets can become more expensive. MySQL chooses plans based on data and statistics. See docs/performance-results.txt for measured plans and observations, not a performance guarantee.

Changes, migrations and assumptions

Branch updates, branch soft deletions and whitelist changes are recorded with the actor, time, action and relevant before/after details in the same transaction. Direct database changes do not automatically create application audit records.

The default branch policy rejects deletion while non-deleted stores exist. The optional HIDE_STORES policy hides child stores through the branch visibility rule. Flyway applies versioned schema and sample-data migrations; Hibernate validates the resulting schema. The service does not synchronize with a real master-data system.